

LAPORAN MILESTONE 3

TUGAS BESAR TEORI BAHASA FORMAL DAN AUTOMATA

IF2224 - Teori Bahasa Formal dan Automata

Kelompok ZZZ - JadiApaArtiHidup?



Anggota Kelompok:

Ahmad Syafiq	13523135
Frederiko Eldad Mugiyono	13523147
Naufarrel Zhafif Abhista	13523159
Hasri Fayadh Muqaffa	13523156
I Made Wiweka Putera	13523160

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2025

Daftar Isi

Daftar Isi.....	2
Landasan Teori.....	3
1. Analisis Semantik.....	3
2. Abstract Syntax Tree.....	3
a. Syntax-Directed Translation Scheme.....	3
b. Type and Scope Checking.....	4
3. Symbol Table.....	4
Perancangan dan Implementasi.....	5
1. Symbol Table.....	5
a. tab.....	5
b. atab.....	6
c. btab.....	6
d. display.....	7
2. Decorated AST.....	7
3. Struktur Proyek.....	8
4. Alur Kerja Analyzer.....	10
5. Penjelasan Modul Implementasi.....	11
Pengujian.....	13
1. Pengujian.....	13
Tabel 1.1. Hasil Pengujian TC1.....	15
Tabel 1.2. Hasil Pengujian TC2.....	17
Tabel 1.3. Hasil Pengujian TC3.....	20
Tabel 1.4. Hasil Pengujian TC4.....	21
Tabel 1.5. Hasil Pengujian TC5.....	25
2. Analisis Hasil Pengujian.....	25
Kesimpulan dan Saran.....	27
1. Kesimpulan.....	27
2. Saran.....	27
Lampiran.....	29
Referensi.....	30

Landasan Teori

1. Analisis Semantik

Analisis semantik adalah fase ketiga dalam proses kompilasi, yang terjadi setelah *lexical analysis* (scanning) dan *syntax analysis* (parsing). Jika parser bertanggung jawab untuk memastikan bahwa struktur kode mematuhi aturan tata bahasa (*grammar*) bebas konteks, maka analisis semantik bertanggung jawab untuk memeriksa "makna" atau konsistensi dari struktur tersebut.

Tujuan utama dari analisis semantik adalah:

- Memastikan operasi dilakukan pada tipe data yang kompatibel (misalnya, tidak menjumlahkan integer dengan string).
- Memastikan variabel atau fungsi dideklarasikan sebelum digunakan.
- Memastikan identifier diakses dalam lingkup visibilitas yang benar.

Pada Pascal-S, analisis semantik dilakukan dengan menelusuri *Parse Tree* dan memvalidasi aturan-aturan konteks yang tidak dapat ditangani oleh *Context-Free Grammar* (CFG), yang akan menghasilkan *Decorated Abstract Syntax Tree*.

2. Abstract Syntax Tree

a. Syntax-Directed Translation Scheme

Untuk membangun AST, digunakan metode *Syntax-Directed Translation Scheme* (SDTS). SDTS adalah formalisme yang mengasosiasikan aturan produksi grammar dengan *semantic actions* (aksi semantik).

Dalam implementasi ini, setiap kali parser berhasil mengenali suatu aturan produksi (misalnya *<assignment-statement>*), sebuah aksi semantik akan dipicu untuk membentuk *node* baru pada AST. Berbeda dengan *Parse Tree* (*Concrete Syntax Tree*) yang memuat semua detail sintaksis (termasuk tanda baca seperti titik koma atau kurung), AST hanya menyimpan informasi struktural yang penting untuk eksekusi program.

b. Type and Scope Checking

Setelah AST terbentuk (atau terbentuk secara *on-the-fly*), dilakukan proses validasi tipe dan lingkup. Proses ini umumnya dilakukan dengan penelusuran (*traversal*) pohon secara *Depth-First*.

- **Type Checking:** Setiap node ekspresi akan dikalkulasi tipe datanya. Aturan inferensi tipe diterapkan (misalnya, hasil penjumlahan dua integer adalah integer). Jika ditemukan ketidakcocokan (misalnya *boolean + integer*), kompiler akan membangkitkan *semantic error*.
- **Scope Checking:** Kompiler memelihara struktur data *stack* dari *Symbol Table* untuk melacak lingkup aktif. Ketika memasuki blok baru (misalnya prosedur), lingkup baru di-*push* ke *stack*. Ketika keluar blok, lingkup di-*pop*. Pencarian identifier dilakukan dari lingkup terdalam hingga ke lingkup global.

3. Symbol Table

Symbol Table adalah struktur data vital dalam analisis semantik yang digunakan untuk menyimpan informasi mengenai identifier (variabel, konstanta, prosedur, fungsi, tipe, dll) yang dideklarasikan dalam program.

Untuk bahasa Pascal-S, *Symbol Table* dirancang khusus untuk menangani fitur bahasa seperti *nested procedures* dan *arrays*. Informasi yang disimpan mencakup:

- Nama Identifier
- Jenis Objek (Variabel, Fungsi, dll.)
- Tipe Data
- Level Kedalaman (Lexical Level)
- Alamat/Offset Memori

Struktur data ini memungkinkan *analyzer* untuk melakukan *lookup* (pencarian) cepat guna memvalidasi apakah suatu variabel telah dideklarasikan dan apakah penggunaannya sesuai dengan tipenya.

Perancangan dan Implementasi

1. Symbol Table

a. tab

tab		
Nama Teoritis	Implementasi Kode	Keterangan
identifiers	name	Nama identifier (misalnya nama variabel, konstanta, tipe, prosedur, fungsi). Indeks dimulai dari 29 karena ada reserved identifier.
link	link	Pointer/indeks ke identifier sebelumnya dalam scope yang sama. Digunakan untuk manajemen scope (linked list per blok).
obj	obj	Kelas objek yang dinumerasi: konstanta, variabel, tipe, prosedur, fungsi, dll.
type	typ	Tipe dasar dari identifier. Implementasi kami menggunakan konstanta numerik: <pre>pub const TYP_NOTYPE: usize = 0; pub const TYP_INT: usize = 1; pub const TYP_REAL: usize = 2; pub const TYP_BOOL: usize = 3; pub const TYP_CHAR: usize = 4; pub const TYP_STRING: usize = 5;</pre>
ref	ref_idx	Pointer ke atab (jika array) atau btab (jika proc/func)
nrm	normal	True = normal/value, false = reference (var)
lev	level	Scope level (0=global, 1=main, dst)
adr	adr	Offset memori atau value konstanta

b. atab

atab		
Nama Teoritis	Implementasi Kode	Keterangan
arrays	-	Indeks entri array, tidak menggunakan kolom eksplisit untuk indeks karena menggunakan indeks bawaan dari entri tersebut.
xtyp	xtyp	Tipe indeks
etyp	etyp	Tipe elemen
low	low	Batas bawah array
high	high	Batas atas array
elsz	elsz	Ukuran elemen array
size	size	Total ukuran array

c. btab

btab		
Nama Teoritis	Implementasi Kode	Keterangan
blocks	-	Indeks entri block, tidak menggunakan kolom eksplisit untuk indeks karena menggunakan indeks bawaan dari entri tersebut.
last	last	Pointer ke identifier terakhir yang dideklarasikan di blok ini
lpar	lpar	Pointer ke parameter terakhir
psze	psze	Parameter size (total ukuran parameter)

vsze	vsze	Variable size (total ukuran variabel lokal)
------	------	---

d. display

Untuk menangani *lexical scoping* (akses variabel global dari prosedur lokal), digunakan vektor `display`. `display[i]` menyimpan indeks `btabs` yang sedang aktif pada level kedalaman `i`.

2. Decorated AST

Struktur data ini didefinisikan dalam `ast.rs` dengan `ProgramAST` sebagai `root` yang membawahi:

- **Decl (Declarations):** Enum yang menangani deklarasi `Constant`, `Type`, `Variable`, `Procedure`, dan `Function`.
- **Stmt (Statements):** Enum untuk logika program seperti `Assignment`, `If`, `While`, `For`, dan `ProcedureCall`.
- **Expr (Expressions):** Struct yang merepresentasikan ekspresi (aritmatika, logika, akses variabel).

Kunci dari *Decorated AST* terletak pada struct `Expr` yang memiliki field khusus bernama `annotation`. Field ini bertipe `SemanticData` yang menyimpan hasil analisis semantik:

```
#[derive(Debug, Clone, PartialEq)]
pub struct SemanticData {
    pub type_kind: Option<TypeKind>, // Hasil evaluasi tipe (misal: Integer, Boolean)
    pub tab_index: Option<usize>,    // Indeks langsung ke Symbol Table (untuk Variabel/Fungsi)
    pub is_const: bool,              // Penanda apakah nilai bersifat immutable (konstanta)
}
```

Dengan struktur ini, informasi tipe (`TypeKind`) dan lokasi memori (`tab_index`) tersimpan langsung di pohon sintaks, sehingga tahapan *Code Generation* nantinya tidak perlu melakukan pencarian simbol ulang.

3. Struktur Proyek

Proyek ini dan struktur folder diatur mengikuti konvensi standar dari Cargo. Adapun struktur dari proyek ini adalah sebagai berikut.

```
ZZZ-Tubes-IF2224
├── Cargo.lock
├── Cargo.toml
├── config
│   └── dfa.json
├── doc
│   ├── Diagram-1-ZZZ.png
│   ├── Laporan-1-ZZZ.pdf
│   └── Laporan-2-ZZZ.pdf
├── examples
│   ├── comment_test.pas
│   ├── comprehensive_test.pas
│   └── hello.pas
├── LICENSE
├── README.md
├── scripts
├── src
│   ├── code_generator
│   ├── interpreter
│   ├── lexer
│   │   ├── dfa.rs
│   │   ├── lexer.rs
│   │   ├── mod.rs
│   │   └── token_types.rs
│   ├── main.rs
│   ├── parser
│   │   ├── declarations.rs
│   │   ├── error.rs
│   │   ├── expressions.rs
│   │   ├── mod.rs
│   │   ├── parser.rs
│   │   └── parse_tree.rs
```



```

|   |   ├── statements.rs
|   |   └── tree_printer.rs
|   ├── semantic_analyzer
|   |   ├── analyzer.rs
|   |   ├── ast
|   |   |   ├── ast_builder.rs
|   |   |   ├── ast.rs
|   |   |   └── mod.rs
|   |   ├── error.rs
|   |   ├── mod.rs
|   |   └── tab.rs
|   └── utils
└── test
    ├── integration
    ├── milestone-1
    ├── milestone-2
    ├── milestone-3
    ├── milestone-4
    └── milestone-5

```

Keterangan:

- `config/`: Berisi file `dfa.json`. File tersebut menyimpan aturan DFA yang akan dibaca oleh program rust untuk mengetahui cara mengenali token.
- `doc/`: Berisi dokumen spesifikasi dan laporan dari tiap milestone.
- `examples/`: Berisi contoh `source code` Pascal-S (`.pas`) yang digunakan sebagai masukan untuk compiler dan memastikan *lexical analyzer* berfungsi dengan benar.
- `src/`: Berisi `source code` utama dari tugas besar ini. Terdapat beberapa folder di dalamnya. Untuk milestone ini, fokus utamanya berada pada folder `parser`. Selain itu, terdapat file `main.rs` yang menjadi *entry point* dari program.
- `target/`: Berisi hasil kompilasi dari program yang dibuat oleh Cargo.
- `test/`: Berisi file *input* dan *output* dari file uji.

- Cargo.toml: Berisi manifest dari proyek ini yang mendefinisikan proyek, versi dan dependensi atau *library* eksternal yang digunakan.
- README.md: Berisi dokumentasi dari proyek.

4. Alur Kerja Analyzer

Logika utama analisis semantik diimplementasikan pada struct `SemanticAnalyzer` yang bekerja dengan pola **Visitor Pattern**. Analyzer menelusuri AST secara *Depth-First* dan melakukan validasi bertahap.

Alur kerjanya adalah sebagai berikut:

1. **Inisialisasi:** Membentuk `SymbolTable` dan mengisi identifier bawaan (*standard types* dan *procedures* seperti `integer`, `writeln`).
2. **Visit Program:** Masuk ke scope global, melakukan analisis pada bagian deklarasi (`visit_decls`), kemudian masuk ke badan program utama (`visit_block`).
3. **Visit Declarations:**
 - a. Mendaftarkan variabel/konstanta/prosedur baru ke `SymbolTable`.
 - b. Memeriksa duplikasi nama (*Duplicate Identifier Error*).
 - c. Untuk Prosedur/Fungsi: Membuat scope baru (`enter_scope`), mendaftarkan parameter, lalu memproses blok lokalnya.
4. **Visit Statements:**
 - a. **Assignment:** Memastikan target bukan konstanta (*AssignmentToConstant Error*) dan tipe data nilai kompatibel dengan tipe variabel target.
 - b. **If/While:** Memastikan kondisi ekspresi menghasilkan tipe `Boolean`.
 - c. **Procedure Call:** Memvalidasi keberadaan prosedur, jumlah argumen, dan kesesuaian tipe argumen.
5. **Visit Expressions:**
 - a. Melakukan inferensi tipe (misal: `Integer + Real` menghasilkan `Real`).
 - b. Memastikan operand sesuai dengan operator (misal: `AND` hanya untuk `Boolean`).
 - c. Mengisi field `annotation` pada node AST dengan hasil tipe data.

5. Penjelasan Modul Implementasi

Kode dibagi menjadi beberapa modul:

1. `src/semantic_analyzer/analyzer.rs`

- **Peran:** Modul inti yang menjalankan logika validasi semantik.
- **Komponen Utama:** Struct `SemanticAnalyzer` yang mengimplementasikan fungsi traversal (`visit_program`, `visit_stmt`, `visit_expr`).
- **Fungsi:** Mengelola transisi scope, melakukan pengecekan tipe (*type checking*), dan melempar *error* jika ditemukan pelanggaran aturan semantik.

2. `src/semantic_analyzer/tab.rs`

- **Peran:** Implementasi struktur data *Symbol Table*.
- **Komponen Utama:** Struct `SymbolTable`, `TabEntry` (identifer), `BTabEntry` (blok), dan `ATabEntry` (array).
- **Fungsi:** Menyediakan operasi low-level seperti `enter()` (menambah simbol), `find()` (mencari simbol dengan aturan scope), serta `make_block()` dan `make_array()`.

3. `src/semantic_analyzer/ast/ast_builder.rs`

- **Peran:** Penerjemah dari Concrete Syntax Tree (Parse Tree) ke Abstract Syntax Tree (AST).
- **Komponen Utama:** Struct `ASTBuilder` dengan fungsi `build()`.
- **Fungsi:** Mengonversi token mentah dari parser menjadi struktur logis `ProgramAST`, menangani pemetaan operator, dan menyederhanakan struktur loop/percabangan.

4. `src/semantic_analyzer/ast/ast.rs`

- **Peran:** Definisi tipe data untuk struktur AST.
- **Komponen Utama:** Struct `ProgramAST`, `Expr`, `SemanticData`, serta Enum `Decl`, `Stmt`, dan `TypeKind`.

- **Fungsi:** Sebagai kontrak data yang digunakan bersama oleh *builder* dan *analyzer*.

5. `src/semantic_analyzer/error.rs`

- **Peran:** Definisi dan manajemen kesalahan semantik.
- **Komponen Utama:** Enum `SemanticErrorKind` dan struct `SemanticError`.
- **Fungsi:** Menyediakan jenis error yang spesifik seperti `TypeMismatch`, `UndefinedIdentifier`, `ArgumentCountMismatch`, dan `NotArray` agar pesan kesalahan mudah dipahami oleh pengguna.

Pengujian

1. Pengujian

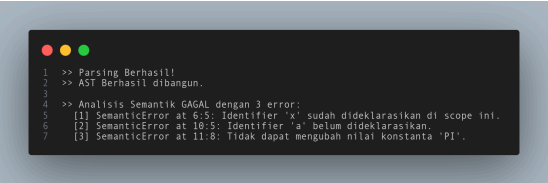
Pada bagian ini, dilakukan serangkaian pengujian terhadap *semantic analyzer* untuk memvalidasi kemampuannya dalam memverifikasi struktur semantik program pada bahasa Pascal. Pengujian dilakukan dengan memberikan lima berkas masukan **.pas** yang berbeda. Setiap pengujian disajikan dalam tabel di bawah, lengkap dengan *input*, *output*, dan analisisnya.

Tabel 1.1. Hasil Pengujian TC1

Test Case 1	
Passing var parameter vs nrm parameter test1_var.pas	
Input	Output
<pre>1 program CekReference; 2 variabel 3 global: integer; 4 5 prosedur Tukar(var a: integer; b: integer; 6 mulai 7 a := b; 8 b := 0; 9 selesai; 10 11 mulai 12 global := 10; 13 Tukar(global, 5); 14 selesai.</pre>	<pre>1 >> Parsing Berhasil! 2 >> AST Berhasil dibangun. 3 >> Analisis Semantik BERHASIL! 4 >> Symbol Table State: 5 6 >> Symbol Table Dump 7 8 ----- TAB (Identifiers) ----- 9 Idx Name Link Obj Typ Ref Mem Len Addr 10 ----- ----- ----- ----- ----- ----- ----- ----- ----- 11 1 0 0 0 0 0 0 0 0 12 2 0 0 0 0 0 0 0 0 13 3 0 0 0 0 0 0 0 0 14 4 0 0 0 0 0 0 0 0 15 5 0 0 0 0 0 0 0 0 16 6 0 0 0 0 0 0 0 0 17 7 0 0 0 0 0 0 0 0 18 8 0 0 0 0 0 0 0 0 19 9 0 0 0 0 0 0 0 0 20 10 0 0 0 0 0 0 0 0 21 11 0 0 0 0 0 0 0 0 22 12 0 0 0 0 0 0 0 0 23 13 0 0 0 0 0 0 0 0 24 14 0 0 0 0 0 0 0 0 25 15 0 0 0 0 0 0 0 0 26 16 0 0 0 0 0 0 0 0 27 17 0 0 0 0 0 0 0 0 28 18 0 0 0 0 0 0 0 0 29 19 0 0 0 0 0 0 0 0 30 20 0 0 0 0 0 0 0 0 31 21 0 0 0 0 0 0 0 0 32 22 0 0 0 0 0 0 0 0 33 23 0 0 0 0 0 0 0 0 34 24 0 0 0 0 0 0 0 0 35 25 0 0 0 0 0 0 0 0 36 26 0 0 0 0 0 0 0 0 37 27 0 0 0 0 0 0 0 0 38 28 0 0 0 0 0 0 0 0 39 29 0 0 0 0 0 0 0 0 40 30 0 0 0 0 0 0 0 0 41 31 0 0 0 0 0 0 0 0 42 32 0 0 0 0 0 0 0 0 43 33 0 0 0 0 0 0 0 0 44 34 0 0 0 0 0 0 0 0 45 35 0 0 0 0 0 0 0 0 46 36 0 0 0 0 0 0 0 0 47 37 0 0 0 0 0 0 0 0 48 38 0 0 0 0 0 0 0 0 49 39 0 0 0 0 0 0 0 0 50 40 0 0 0 0 0 0 0 0 51 41 0 0 0 0 0 0 0 0 52 42 0 0 0 0 0 0 0 0 53 43 0 0 0 0 0 0 0 0 54 55 ----- BTAB (Blocks) ----- 56 Idx Last LPar PSize WSize 57 ----- ----- ----- ----- ----- 58 0 0 0 0 0 59 1 41 0 0 1 60 2 0 0 0 0 61 62 63 [ATAB is empty] 64 >> Decorated AST: 65 ProgramNode(name: 'CekReference') 66 Declarations 67 VarDecl((global) -> type: Integer 68 ProcedureDecl('Tukar') 69 Params 70 var a: Integer 71 b: Integer 72 Declarations 73 Body 74 Assign('a' := 'b') -> tab_index:42, type: Integer 75 target Variable(a) -> tab_index:43, type: Integer 76 value Variable(b) -> tab_index:43, type: Integer 77 Assign('b' := 0) -> tab_index:43, type: Integer 78 target Variable(b) -> tab_index:43, type: Integer 79 value Number(0) -> type: Integer 80 Block 81 Assign('global' := 10) -> tab_index:48, type: Integer 82 target Variable(global) -> tab_index:48, type: Integer 83 value Number(10) -> type: Integer 84 call('Tukar') 85 arg(0) Variable('global') -> tab_index:48, type: Integer 86 arg(1) Number(5) -> type: Integer 87 88</pre>

Analisis
<i>Analyzer</i> berhasil melakukan <i>semantic analysis</i> dan juga membedakan <i>pass-by-value</i> dan <i>pass-by-parameter</i> .

Tabel 1.2. Hasil Pengujian TC2

Test Case 2	
Test Error <i>test2_error.pas</i>	
Input	Output
 <pre>1 program FailDecl; 2 konstanta 3 PI = 3.14; 4 variabel 5 x, y : integer; 6 x : boolean; 7 z : integer; 8 9 mulai 10 a := 10; 11 PI := 5.0; 12 selesai.</pre>	 <pre>>> Parsing Berhasil! >> AST Berhasil dibangun. >> Analisis Semantik GAGAL dengan 3 error: [1] SemanticError at 6:5: Identifier 'x' sudah dideklarasikan di scope ini. [2] SemanticError at 10:5: Identifier 'a' belum dideklarasikan. [3] SemanticError at 11:8: Tidak dapat mengubah nilai konstanta 'PI'.</pre>
Analisis	
<i>Analyzer</i> berhasil <i>handle</i> error pada deklarasi variabel.	

Tabel 1.3. Hasil Pengujian TC3

Test Case 3	
Test Error 2 <i>test3_error_2.pas</i>	
Input	Output

<pre>1 program IndexIteratorConst; 2 konstanta 3 MAX_LIMIT = 50; 4 5 variabel 6 global_counter : integer; 7 8 prosedur UbahNilai(var target: integer); 9 mulai 10 target := target + 1; 11 selesai; 12 13 prosedur TestLogic(); 14 variabel 15 data : larik [1..5] dari integer; 16 mulai 17 untuk global_counter := 1 ke 3 lakukan 18 data[global_counter] := global_counter * 10; 19 20 data[6] := 999; 21 data[0] := -1; 22 selesai; 23 24 mulai 25 TestLogic(); 26 UbahNilai(MAX_LIMIT); 27 selesai.</pre>	<pre>1 >> Parsing Berhasil! 2 >> AST Berhasil dibangun. 3 >> Analisis Semantik gagal dengan 4 error: 4 [1] SemanticError at 17:5: Iterator 'global_counter' harus berupa variabel lokal bertipe ordinal. 5 [2] SemanticError at 18:28: Array index 6 di luar jangkauan [1..5] 6 [3] SemanticError at 21:28: Array index 0 di luar jangkauan [1..5] 7 [4] SemanticError at 16:25: Argument 1 ts a Constant, cannot be passed to 'var' parameter 8</pre>
Analisis	
<i>Analyzer berhasil handle error pada index out of bounds, assign constant, dan tipe iterator.</i>	

Tabel 1.4. Hasil Pengujian TC4

Test Case 4	
Nested Matrix <code>test4_edge.pas</code>	
Input	Output

Analisis
<i>Analyzer</i> berhasil meng-handle pemanggilan fungsi dalam array call.

Tabel 1.5. Hasil Pengujian TC5

Test Case 5	
Rekursi dan Scope Test <code>test5_scope_recursion.pas</code>	
<i>Input</i>	<i>Output</i>

Analyzer berhasil handle rekursi (contohnya disini Fibonacci)

2. Analisis Hasil Pengujian

Secara keseluruhan, hasil dari kelima kasus uji menunjukkan bahwa *Semantic Analyzer* yang diimplementasikan telah berfungsi sesuai dengan spesifikasi Milestone 3. Analisis mendalam dapat dirangkum sebagai berikut:

1. Pengujian pada TC2 dan TC5 membuktikan bahwa *Symbol Table* (terdiri dari tab, btab, dan atab) dikelola dengan benar. Implementasi *Display Stack* dan atribut *level* memungkinkan analyzer membedakan variabel global dan lokal, serta menangani *shadowing* variabel dengan tepat. Fitur *lookup* identifier bekerja secara hierarkis, mencari dari scope terdalam hingga ke global.
2. Kasus uji TC1 dan TC4 menunjukkan *strong typing*. Analyzer mampu memvalidasi kompatibilitas tipe pada operasi aritmatika, logika, *assignment*, dan *passing* parameter. Ini termasuk penanganan tipe data kompleks seperti Array dan Custom Type, serta memastikan argumen fungsi sesuai dengan definisi parameternya.
3. Berbeda dengan parser yang hanya mengecek sintaks, TC3 membuktikan bahwa analyzer mampu mengevaluasi batasan logis program. Implementasi helper *check_array_bounds* memungkinkan deteksi *Index Out of Bounds* secara statis (compile-time) untuk berbagai tipe ordinal. Selain itu, aturan semantik spesifik seperti larangan modifikasi konstanta dan kewajiban iterator lokal pada loop *for* telah terimplementasi dengan baik.
4. Keberhasilan TC5 memvalidasi perbaikan logika pada resolusi nama fungsi. Analyzer dapat membedakan antara variabel lokal yang memiliki nama sama dengan fungsi (untuk *return value*) dan pemanggilan fungsi rekursif itu sendiri dengan menelusuri stack scope ke atas.
5. Seluruh pengujian menghasilkan *Decorated AST*. Setiap node ekspresi dan deklarasi kini memiliki anotasi tipe data (*type_kind*) dan referensi ke tabel simbol (*tab_index*), yang siap digunakan untuk tahapan selanjutnya (*Code Generation*).

Berdasarkan analisis tersebut, dapat disimpulkan bahwa implementasi *Semantic Analysis* telah memenuhi seluruh kriteria keberhasilan. Program tidak hanya menerima kode yang benar secara sintaks, tetapi juga menjamin kebenaran makna dan keamanan tipe data sebelum kode dieksekusi atau dikompilasi lebih lanjut.

Kesimpulan dan Saran

1. Kesimpulan

Berdasarkan tahapan perancangan, implementasi, dan pengujian yang telah dilakukan untuk Milestone 3, dapat disimpulkan bahwa:

1. Modul *Semantic Analyzer* telah berhasil diimplementasikan untuk melakukan validasi makna terhadap kode program Pascal-S, melengkapi fungsionalitas parser dari milestone sebelumnya.
2. Sistem *Type Checking* dan *Scope Checking* berfungsi dengan baik menggunakan struktur *Symbol Table* yang hierarkis (tab, btab, atab). Compiler mampu mendeteksi kesalahan semantik umum seperti ketidakcocokan tipe (*Type Mismatch*) atau penggunaan variabel tanpa deklarasi (*Undefined Identifier*).
3. Proses transformasi dari *Parse Tree* ke *Decorated Abstract Syntax Tree* (AST) berjalan lancar. Struktur AST yang dihasilkan lebih sederhana namun kaya informasi, memuat anotasi tipe dan referensi simbol yang diperlukan untuk tahap selanjutnya.
4. Implementasi *Symbol Table* yang mengacu pada desain referensi Pascal-S (menggunakan link antar identifier dalam satu scope) terbukti efektif menangani *nested scopes* pada prosedur dan fungsi.

2. Saran

Meskipun implementasi Milestone 3 telah memenuhi spesifikasi, terdapat beberapa saran untuk pengembangan selanjutnya menuju tahap *Code Generation*:

1. **Optimasi Symbol Table:** Saat ini pencarian identifier (*lookup*) dilakukan secara linear pada *linked list* di setiap level scope. Untuk program dengan jumlah variabel yang sangat besar, struktur data ini dapat dioptimasi menggunakan *Hash Map* untuk setiap scope guna meningkatkan performa lookup.
2. **Persiapan Code Generation:** Kalkulasi atribut memori (seperti *stack offset* dan ukuran tipe data) pada *Symbol Table* perlu diverifikasi lebih lanjut ketepatannya. Hal ini krusial untuk memastikan *addressing* variabel pada tahap *Code*

Generation nantinya akurat sesuai dengan arsitektur mesin target (Interpreter Pascal-S).

Lampiran

- Link Release Repository:
<https://github.com/reletz/ZZZ-Tubes-IF2224/releases/tag/v0.3.2>
- Link Repository: <https://github.com/reletz/ZZZ-Tubes-IF2224>
- Tabel Pembagian Tugas sebagai berikut:

No	Nim	Nama	Tugas	Persentase
1	13523135	Ahmad Syafiq	• Analyzer - Array dan Function Calls • Semantic Error	17%
2	13523147	Frederiko Eldad Mugiyono	• Semantic Error • Symbol Table - atab, btab, Bitpacking adr, print_tables (Human Readable).	19%
3	13523149	Naufarrel Zhafif Abhista	• Decorated AST - Node, Builder, Printer • Symbol Table - tab, atab, btab • Analyzer - Recursive Call	23%
4	13523156	Hasri Fayadh Muqaffa	• Parser - var Token • Analyzer - Expressions, Statement, Flow	25%

			Control, Type Checker • Symbol Table - Reserved words	
5	13523160	I Made Wiweka Putera	• Analyzer - Declarations dan Scope	16%

Referensi

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compilers: Principles, techniques, & tools* (2nd ed.). Pearson/Addison Wesley.
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introduction to automata theory, languages, and computation* (3rd ed.). Pearson/Addison Wesley.
- Klabnik, S., & Nichols, C. (2023). *The Rust programming language* (2nd ed.). No Starch Press.